

Complete Guide: Crop Disease Dataset Training using Roboflow & YOLOv8

1. Objective

This guide explains how to create, annotate, and train a high-quality crop disease dataset using Roboflow and YOLOv8 for AI-based plant disease detection applications.

2. Define Scope

Select 1–2 crops and define disease classes clearly.

- 1 Choose crops (e.g., Tomato, Potato, Chili)
- 2 Define 3–5 disease classes per crop
- 3 Include 'Healthy' class
- 4 Avoid too many classes initially

3. Data Collection

Collect diverse and real-world images.

- 1 Minimum 300–500 images per class
- 2 Use mobile camera, field images, Kaggle datasets
- 3 Capture images in different lighting conditions
- 4 Include zoomed and full-leaf images

4. Data Quality Rules

- 1 Images must be clear (avoid extreme blur)
- 2 Disease should be visible
- 3 Avoid duplicate images
- 4 Use real field conditions where possible

5. Upload Dataset to Roboflow

- 1 Create a new project in Roboflow
- 2 Select 'Object Detection' project type
- 3 Upload all collected images
- 4 Organize images properly

6. Annotation Process

Annotate infected regions using bounding boxes.

- 1 Draw boxes around diseased areas only
- 2 Do not mark the entire leaf unless necessary
- 3 Assign correct class labels
- 4 Maintain consistent naming

7. Annotation Best Practices

- 1 Keep bounding boxes tight
- 2 Avoid overlapping incorrect regions
- 3 Ensure labeling consistency
- 4 Review annotations after completion

8. Dataset Splitting

- 1 Training: 70%
- 2 Validation: 20%
- 3 Testing: 10%

9. Data Augmentation (Roboflow)

- 1 Flip images (horizontal/vertical)
- 2 Adjust brightness and contrast
- 3 Apply slight rotations
- 4 Do not over-augment

10. Export Dataset

- 1 Select YOLOv8 format
- 2 Download dataset
- 3 Ensure data.yaml file is included

11. YOLOv8 Training

Install and train the model using Ultralytics.

- 1 Install: `pip install ultralytics`
- 2 Train command:
- 3 `yolo detect train data=data.yaml model=yolov8n.pt epochs=50 imgsz=640`

12. Model Evaluation

- 1 Check mAP score
- 2 Evaluate precision and recall
- 3 Test on new images
- 4 Identify incorrect predictions

13. Model Improvement

- 1 Add more images for weak classes
- 2 Fix incorrect annotations
- 3 Retrain the model
- 4 Repeat until accuracy improves

14. Export Model for Application

- 1 Export to TFLite format
- 2 Use: `yolo export model=best.pt format=tflite`
- 3 Prepare for mobile integration

15. Common Mistakes to Avoid

- 1 Poor annotations
- 2 Unbalanced dataset
- 3 Too many classes
- 4 Low-quality images

16. Final Output

A trained YOLOv8 model (best.pt or TFLite) ready for integration into the mobile application.